Day 149: ARIMA from Scratch: Fitting, Choosing and Forecasting on Real Data

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 22, Day 3 | Complete study guide

Watch: ARIMA & SARIMA Made Easy: Forecasting for Beginners (Prof. Ryan Ahmed) | Coding ARMA Model: Time Series Talk (ritvikmath)

Script: `day149_arima_from_scratch.py` · every number in this guide is copied by program from `day149_run_output.txt`, the real output of that script.

Day 148 made CO2 stationary by differencing it; the differenced series still has memory. Today models that memory with ARIMA, written from scratch: the error recursion, a Levenberg-Marquardt fit by conditional least squares (CSS), forecasts built back up through the differences, and their intervals. Our forecasts match statsmodels' Kalman filter to 2.2e-07 for the same coefficients. The model chosen by AIC on the training years, the 'airline' model, forecast 2000–2001 with a mean error of 0.412 ppm, against 0.591 for Day 147's best. The fit itself differs from statsmodels' exact-likelihood fit (seasonal coefficient -0.6902 against -0.8480), and the lesson measures why. On the electrical-equipment orders the model wins one month ahead and loses a year ahead, and nothing foresaw the 2008–09 crash.

0. Where today fits

DEFINITION — start here, no prior knowledge assumed

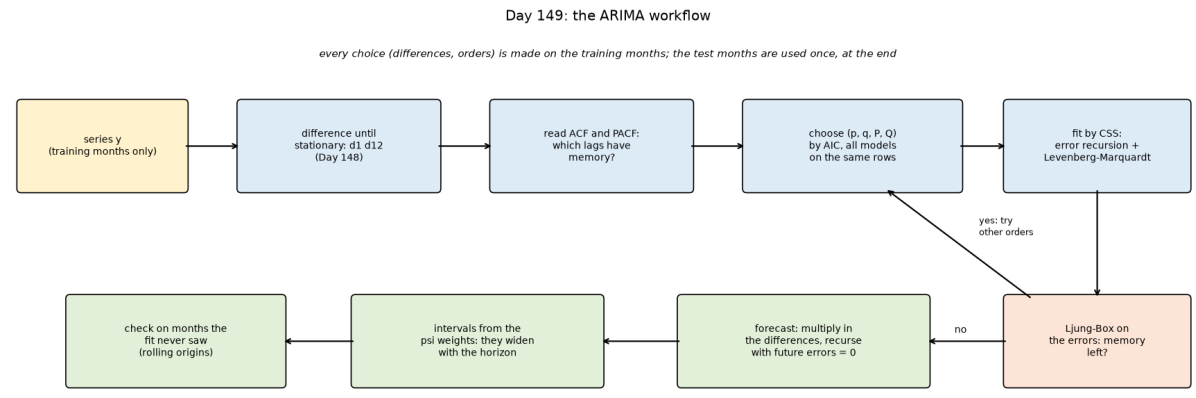
An **autoregressive (AR)** term makes each value depend on earlier values; a **moving-average (MA)** term makes it depend on earlier *surprises*, the errors the model made. **ARIMA(p, d, q)** is p AR terms and q MA terms applied to a series differenced d times. The **seasonal** version adds terms twelve months apart: (P, D, Q) at lag 12.

Day 148 found d = 1 and D = 1 for CO2. Today chooses p, q, P and Q, fits the coefficients and forecasts.

REAL-WORLD SCENARIO

A shop forecasting next month's sales starts from last month (AR) and adjusts for how wrong last month's forecast was (MA): if it under-forecast, some of that surprise will carry on. It also remembers the same month last year (the seasonal terms).

SYSTEM DIAGRAM



The ARIMA workflow: every choice on the training months, the test months used once.

Today's part for the week's build

`day149_exercise.py` is today's parts (the partial autocorrelation, the seasonal ARMA polynomials, the error recursion, a Levenberg-Marquardt solver, the CSS fit, the differencing polynomial, forecasts, the psi weights and the Ljung-Box statistic), checked by hand: a partial autocorrelation, the airline polynomial, three errors of an MA(1), a straight-line fit against np.polyfit, and a random walk's forecast and intervals.

ACTUAL OUTPUT

```
pacf of 1, 2, 3, 4, 5 at lag 2: (r2 - r1^2) / (1 - r1^2) = (-0.1 - 0.16) / 0.84 = -0.3095; pacf()
    gives -0.3095
airline MA polynomial (1 - 0.4 B)(1 - 0.6 B^12): lags [0, 1, 12, 13], coefficients [1.0, -0.4,
    -0.6, 0.24] (by hand: 1, -0.4, -0.6, +0.24)
MA(1), theta 0.5, on 1, 2, 3: e = 1, 2 - 0.5 x 1 = 1.5, 3 - 0.5 x 1.5 = 2.25; css_residuals gives
    [1.0, 1.5, 2.25]
a straight line: levenberg_marquardt [3.038182, 1.991515], np.polyfit [3.038182, 1.991515]
(1 - B)(1 - B^12) = 1 - B - B^12 + B^13: full_ar gives lags [0, 1, 12, 13], coefficients [1.0,
    -1.0, -1.0, 1.0]
random walk 1, 2, 5: forecast [5.0, 5.0, 5.0] (the last value), psi weights [1.0, 1.0, 1.0, 1.0]
    (so the interval grows with sqrt(h))
Ljung-Box, one lag, 200 values: n(n + 2) r1^2 / (n - 1) = 0.2803; ljung_box gives 0.2803
```

Data: Day 147's two series, loaded by its parts. CO2 is trained up to 1999-12 and tested on the 24 months of 2000–2001, Day 147's split, so its simple forecasts are the baseline. The orders are used on a log scale (Day 147, section 4). Simulated series with known coefficients serve as controls.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. AR and MA signatures: the ACF and the PACF

DEFINITION — start here, no prior knowledge assumed

The **partial autocorrelation** (PACF) at lag k is the correlation k steps apart after removing what lags 1 to k–1 already explain. An AR(p) series has a PACF that stops after lag p and an ACF that fades; an MA(q) series is the mirror image: its ACF stops after lag q and its PACF fades. That is how p and q are read off real data.

REAL-WORLD SCENARIO

Your mood today depends on yesterday's mood, which depended on the day before: an AR chain, where the influence of a week ago passes through every day between. A single bad night's sleep affects today and tomorrow and then is gone: an MA shock.

MATHEMATICS

$AR(p)$: $y_t = \phi_1 y_{t-1} + \ldots + \phi_p y_{t-p} + e_t$ $\quad MA(q)$: $y_t = e_t + \theta_1 e_{t-1} + \ldots + \theta_q e_{t-q}$

PACF by Durbin-Levinson: $a = (r_k - \Sigma\, \phi_j\, r_{k-j}) / (1 - \Sigma\, \phi_j\, r_j)$, then $\phi_j \leftarrow \phi_j - a\, \phi_{k-j}$

IMPLEMENTATION

```
def pacf(x, nlags):
    r = d148.acf(x, nlags); out = [1.0]; phi = np.zeros(0)
    for k in range(1, nlags + 1):
        a = (r[k] - phi @ r[k - 1:0:-1]) / (1 - phi @ r[1:k])
        phi = np.r_[phi - a * phi[::-1], a]; out.append(a)
    return np.array(out)
```

ACTUAL OUTPUT

```
PACF at lag k: the correlation k steps apart after removing what lags 1 ... k-1 already explain
                          series   lag 1   lag 2   lag 3   lag 4   lag 5
AR(2): y = 0.5 y(-1) + 0.3 y(-2) + e, ACF   0.682   0.629   0.487   0.415   0.311
AR(2): y = 0.5 y(-1) + 0.3 y(-2) + e, PACF   0.682   0.307  -0.041   0.002  -0.049
     MA(1): y = e + 0.7 e(-1), ACF   0.470   0.016   0.001  -0.018  -0.011
    MA(1): y = e + 0.7 e(-1), PACF   0.470  -0.264   0.160  -0.125   0.081
theory: AR(2) ACF lag 1 = 0.5 / (1 - 0.3) = 0.714, PACF lag 2 = 0.3, PACF beyond 2 = 0; MA(1) ACF lag 1 = 0.7 / (1 + 0.49) =
    0.470, ACF beyond 1 = 0
largest difference from statsmodels' pacf (Levinson-Durbin), lags 0-24: 4.0e-16
band for 'no correlation' at 500 values: +-0.088; the AR(2)'s PACF stops after lag 2 while its ACF fades;
the MA(1)'s ACF stops after lag 1 while its PACF fades with alternating signs -- the two signatures used to read real data
Actual output from this lesson's run.
```

What it means

The from-scratch PACF matches statsmodels' Levinson-Durbin version (4.0e-16). The AR(2)'s PACF is 0.682 and 0.307 (theory 0.3 at lag 2) and then within the noise band of ±0.088; its ACF fades (0.629, 0.487, ...). The MA(1)'s ACF is 0.470 at lag 1, exactly the theory, and then near 0, while its PACF fades with alternating signs (-0.264, 0.160).

WHY CHOOSE THIS?

Reading the two plots gives a short list of candidate orders before any fitting.

WHY NOT THIS? (TRADEOFFS)

Real data rarely shows clean cut-offs, and mixed ARMA models have both functions fading; AIC (section 3) settles it.

2. Fitting by conditional least squares

DEFINITION — start here, no prior knowledge assumed

Given coefficients, the model's errors can be computed one by one: each error is the value minus what the model predicted from earlier values and earlier errors. The errors before the series starts are unknown and are set to 0; that is the **conditional** in conditional least squares (CSS). The coefficients are then chosen to make the sum of squared errors as small as possible.

The errors depend on the MA coefficients non-linearly (each error feeds into the next), so there is no formula for the best coefficients. **Levenberg-Marquardt** searches for them: it estimates how each error moves with each coefficient, takes a Gauss-Newton step, and shortens the step whenever it would make things worse.

REAL-WORLD SCENARIO

Tuning a radio by ear: turn the dial a little, listen whether the hiss got better or worse, and take bigger turns while it keeps improving and smaller ones when you overshoot.

MATHEMATICS

$a(B)\, w_t = b(B)\, e_t, \quad B = \text{the lag } (B\, w_t = w_{t-1})$

$a(B) = (1 - \phi_1 B - ...)(1 - \Phi_1 B^{12} - ...), \quad b(B) = (1 + \theta_1 B + ...)(1 + \Theta_1 B^{12} + ...)$

$e_t = a(B)\, w_t - (b_1\, e_{t-1} + b_2\, e_{t-2} + ...), \quad \text{minimise } \Sigma\, e_t^2$

$\text{Levenberg-Marquardt step} = -(J^T J + \lambda\, \text{diag}\, J^T J)^{-1}\, J^T e$

IMPLEMENTATION

```
def sarima_polys(params, order):
    p, q, P, Q, s = order
    phi, theta = params[:p], params[p:p + q]
    PHI, THETA = params[p + q:p + q + P], params[p + q + P:p + q + P + Q]
    sa = np.zeros(s * P + 1); sa[0] = 1; sa[s::s] = -PHI
    sb = np.zeros(s * Q + 1); sb[0] = 1; sb[s::s] = THETA
    return np.convolve(np.r_[1.0, -phi], sa), np.convolve(np.r_[1.0, theta], sb)
def css_residuals(w, a, b, start=None):
    r = len(a) - 1 if start is None else start
    u = np.convolve(w, a)[r:len(w)]
    q = len(b) - 1; lags = np.flatnonzero(b[1:]) + 1; bl = b[lags]
    e = np.zeros(q + len(u))
    for t in range(len(u)):
        e[q + t] = u[t] - bl @ e[q + t - lags]
    return e[q:]
```

IMPLEMENTATION

```
def levenberg_marquardt(fun, x0, iters=200, tol=1e-12):
    x = np.array(x0, float); r = fun(x); ssr = r @ r; lam = 1e-3
    for _ in range(iters):
        J = np.empty((len(r), len(x)))
        for j in range(len(x)):
            dx = np.zeros(len(x)); dx[j] = 1e-6 * max(1.0, abs(x[j]))
            J[:, j] = (fun(x + dx) - r) / dx[j]
        g, H = J.T @ r, J.T @ J
        while True:
            step = np.linalg.solve(H + lam * np.diag(np.diag(H) + 1e-12), -g)
            r_new = fun(x + step); ssr_new = r_new @ r_new
            if ssr_new < ssr:
                break
            lam *= 10
            if lam > 1e10:
                return x, ssr
        x, r, lam, gain, ssr = x + step, r_new, lam / 10, ssr - ssr_new, ssr_new
        if gain < tol * ssr:
            break
    return x, ssr
def fit_css(w, order, start=None):
    def res(th):
        return css_residuals(w, *sarima_polys(th, order), start=start)
    x, ssr = levenberg_marquardt(res, np.full(sum(order[:4]), 0.1))
    m = len(res(x))
    return x, ssr / m, m
```

ACTUAL OUTPUT

```
error recursion against scipy's lfilter, at the true coefficients: largest difference 0.0e+00
                                    phi    theta  error variance
                           truth    0.600    0.300           1.000
   ours: CSS, Levenberg-Marquardt   0.5926   0.2670          1.0002   (14 ms)
  scipy least_squares, same errors   0.5926   0.2670
     statsmodels ARIMA, exact MLE   0.5962   0.2680          1.0076
ours against scipy's solver: largest difference 3.8e-09
Actual output from this lesson's run.
```

What it means

The error recursion agrees exactly with scipy's recursive filter (0.0e+00), and our solver lands on the same minimum as scipy's least_squares (3.8e-09). The fit recovered phi 0.5926 and theta 0.2670 against the true 0.6 and 0.3; statsmodels' exact-likelihood fit gives 0.5962 and 0.2680. On this series the two methods agree closely; section 5 shows a case where they do not.

WHY CHOOSE THIS?

CSS is simple, fast (a whole fit in milliseconds) and needs nothing beyond a recursion and a least-squares solver.

WHY NOT THIS? (TRADEOFFS)

Setting the unknown early errors to 0 is an approximation; with MA coefficients near –1 it biases the fit (section 5).

3. Choosing the model for CO2

DEFINITION — start here, no prior knowledge assumed

Three tools, all applied to the training months only. The ACF and PACF of the differenced series suggest which lags matter. **AIC** compares candidate models: it rewards a smaller error variance and charges 2 for each coefficient, so an extra term must earn its place. All candidates are fitted on the same rows, as Day 148 did for the ADF lags. Then the **Ljung-Box** test checks the chosen model's errors: if memory is left in them, the model missed something.

REAL-WORLD SCENARIO

Choosing a recipe: taste suggests what is missing (the ACF), a scorecard that penalises extra ingredients picks the recipe (AIC), and a final tasting checks nothing obvious is still wrong (Ljung-Box).

MATHEMATICS

$$AIC = m\,\log(\sigma^2) + 2k \quad (m \text{ rows, } k \text{ coefficients}) \qquad Q = n(n+2)\,\Sigma_{k=1..h}\, r_k^{\,2} / (n-k), \text{ compared with } \chi^2 \text{ with } h - \text{(fitted) degrees of freedom}$$

IMPLEMENTATION

```
def ljung_box(e, lags, fitted=0):
    r = d148.acf(e, lags); n = len(e)
    return n * (n + 2) * np.sum(r[1:] ** 2 / (n - np.arange(1, lags + 1))), lags - fitted
```

ACTUAL OUTPUT

```
CO2 d1 d12, training months up to 1999-12 (489 values); band for no correlation: +-0.089
   lag       1       2       3      11      12      13      24      36
   ACF  -0.289   0.005  -0.125   0.138  -0.415   0.088  -0.082   0.046
  PACF  -0.289  -0.086  -0.163   0.179  -0.350  -0.147  -0.255  -0.172
AIC = rows x log(error variance) + 2 x coefficients, 36 models (p, q 0-2; P, Q 0-1), rows from 14 on (6.10 s):
  (p, q, P, Q) = (0, 1, 0, 1)   AIC  -1123.04   error variance 0.0932
  (p, q, P, Q) = (0, 1, 1, 1)   AIC  -1121.59   error variance 0.0931
  (p, q, P, Q) = (0, 2, 0, 1)   AIC  -1121.55   error variance 0.0931
  (p, q, P, Q) = (1, 1, 0, 1)   AIC  -1121.38   error variance 0.0932
  (p, q, P, Q) = (1, 2, 0, 1)   AIC  -1120.43   error variance 0.0930
  ... worst: (0, 0, 0, 0)   AIC   -857.16   error variance 0.1645
the airline model itself implies ACF at lags 11 and 13 of theta THETA / ((1 + theta^2)(1 + THETA^2)) = 0.151 with the fitted
    values; lag 3 (-0.125) is outside the band and not explained by it
the next four models add a coefficient each and lower the error variance by at most 0.0003
Ljung-Box, 24 lags: residuals Q 28.52, 22 df, p 0.159 (statsmodels: Q 28.52, p 0.159); d1 d12 before modelling Q 162.32, p
    1.6e-22
Actual output from this lesson's run.
```

What it means

The ACF has two clear spikes, lag 1 (-0.289) and lag 12 (-0.415), while the PACF fades at 12, 24 and 36 (-0.350, -0.255, -0.172): the MA signature, at both lags. AIC agrees: of 36 models, the smallest AIC is (p, q, P, Q) = (0, 1, 0, 1), one MA term at lag 1 and one at lag 12, known as the **airline model**. The next four each add a coefficient and lower the error variance by at most 0.0003.

The airline model itself implies the small spikes at lags 11 and 13 (0.151 with the fitted values). Lag 3 (-0.125) is outside the band and is not explained by it. The Ljung-Box test on the errors finds no clear memory left (Q 28.52, p 0.159, identical to statsmodels'), against Q 162.32 for the differenced series before modelling.

WHY CHOOSE THIS?

The choice is made on the training years only, by a rule, so the test months stay an honest check.

WHY NOT THIS? (TRADEOFFS)

AIC compares models only within the grid tried, and a test that does not reject is not proof that nothing is left.

4. Forecasting 2000–2001

DEFINITION — start here, no prior knowledge assumed

To forecast, multiply the model's AR polynomial by the differences, $(1 - B)(1 - B^{12})$; the result describes CO2 itself, and the same recursion that produced the errors now produces forecasts, with every future error set to 0, its expected value. Each forecast feeds the next.

The forecast's uncertainty grows with the horizon. Writing the model as a sum of future shocks gives the **psi weights**; the error variance h steps ahead is σ^2 times the sum of the first h squared weights.

REAL-WORLD SCENARIO

A weather forecast for tomorrow is fairly sure; for next week it is a range. The psi weights say how fast the range widens.

MATHEMATICS

$$A(B) = a(B)(1 - B)(1 - B^{12}): \quad y_t = -A_1 y_{t-1} - A_2 y_{t-2} - \ldots + b_1 e_{t-1} + \ldots \quad (\text{future } e = 0)$$

$$\psi_j = b_j - \Sigma_{i=1..j} A_i \psi_{j-i}$$

$$\text{variance } h \text{ steps ahead} = \sigma^2 (\psi_0^2 + \ldots + \psi_{h-1}^2); \; 95\% \text{ interval} = \text{forecast} \pm 1.96 \text{ sd}$$

IMPLEMENTATION

```
def full_ar(a, d=1, D=1, s=12):
    for _ in range(d):
        a = np.convolve(a, [1.0, -1.0])
    for _ in range(D):
        a = np.convolve(a, np.r_[1.0, np.zeros(s - 1), -1.0])
    return a
def forecast(y, A, b, e, h):
    n = len(y); x = np.r_[y, np.zeros(h)]
    E = np.r_[np.zeros(n - len(e)), e, np.zeros(h)]
    for t in range(n, n + h):
        x[t] = -A[1:] @ x[t - 1:t - len(A):-1] + b[1:] @ E[t - 1:t - len(b):-1]
    return x[n:]
def psi_weights(A, b, h):
    p = np.zeros(h)
    for j in range(h):
        k = min(j, len(A) - 1)
        p[j] = (b[j] if j < len(b) else 0.0) - A[1:k + 1] @ p[j - k:j][::-1]
    return p
```

ACTUAL OUTPUT

```
airline model, CSS on 489 rows: theta -0.3659, THETA (lag 12) -0.6902, error variance 0.1081
checks: the model on y against forecasting d1 d12 and undoing both differences by hand: 1.7e-13
        statsmodels' forecast with our coefficients: 2.2e-07; interval widths: 4.2e-09
2000-01 to 2001-12, 24 months       MAE (ppm)   bias
  last value (Day 147)                       2.420  -2.207
  same month last year (Day 147)             1.882  -1.882
  trend line, 120 months + season (Day 147)  0.591  -0.591
  airline, ours (CSS)                        0.412  +0.405
  airline, statsmodels (MLE)                 0.359  +0.359
our 95% interval: half-width 0.64 ppm one month ahead, 1.50 at 12, 2.40 at 24; 24 of 24 months inside
both airline fits beat Day 147's best; both overshoot: ours is above the actual value in 23 of 24 months
MLE coefficients: theta -0.3472, THETA -0.8480, error variance 0.0942
Actual output from this lesson's run.
```

What it means

Three checks hold: forecasting y directly equals forecasting d1 d12 and undoing both differences by hand (1.7e-13); statsmodels' Kalman filter, given our coefficients, produces the same forecasts (2.2e-07) and the same interval widths (4.2e-09).

On the test months the airline model's error is 0.412 ppm, against Day 147's 2.420 (last value), 1.882 (same month last year) and 0.591 (a trend line plus the season). statsmodels' own fit does better still (0.359). Both overshoot: ours is above the actual value in 23 of 24 months. Every test month fell inside our 95% interval (24 of 24 months inside), whose half-width grows from 0.64 ppm one month ahead to 2.40 at 24.

WHY CHOOSE THIS?

One recursion gives forecasts and, through the psi weights, honest intervals that widen with the horizon.

WHY NOT THIS? (TRADEOFFS)

24 test months are a single draw; one good result could be luck, which section 6 checks with many origins.

5. CSS against exact likelihood

DEFINITION — start here, no prior knowledge assumed

statsmodels fits the same model by **maximum likelihood** (MLE): instead of setting the early errors to 0, a Kalman filter accounts for not knowing them. On CO2 the two fits differ, mainly in the seasonal coefficient THETA. This section measures why.

REAL-WORLD SCENARIO

Starting a stopwatch late makes every lap time look a little wrong at first; the exact method knows the stopwatch started late and corrects for it.

ACTUAL OUTPUT

```
CSS starts with the unknown earlier errors set to 0; on CO2 the first 13 errors have mean square 0.3736, the rest 0.1009
                 CO2, airline model    theta    THETA
       CSS, first 0 errors left out  -0.3659  -0.6902
      CSS, first 13 errors left out  -0.3435  -0.6902
      CSS, first 26 errors left out  -0.3742  -0.7242
      CSS, first 39 errors left out  -0.3672  -0.7389
            statsmodels, exact MLE  -0.3472  -0.8480
leaving out early errors moves THETA only to -0.7389; the gap to MLE on CO2 is 0.158 with none left out
40 simulated series of 489 values, true theta -0.35 and THETA -0.85:
  mean CSS -0.3600, -0.8090;  mean MLE -0.3597, -0.8597;  CSS's THETA closer to 0 than MLE's in 40 of 40
so CSS does pull THETA towards 0, by 0.051 on average here: less than CO2's gap; the rest is not explained here
Actual output from this lesson's run.
```

What it means

With the early errors set to 0, the first 13 errors on CO2 are inflated (mean square 0.3736, against 0.1009 for the rest). Leaving more and more of them out of the sum moves THETA only from -0.6902 to -0.7389, still far from MLE's -0.8480.

A simulation with a known answer (THETA = –0.85, 40 series of the same length) confirms a real bias: CSS averaged -0.8090, MLE -0.8597, and CSS was closer to 0 in 40 of 40. But that bias, 0.051 on average, is smaller than CO2's gap of 0.158; the rest is not explained here. A likely reason, not tested here, is that CO2's seasonal pattern drifts (Day 147), which the two methods may weigh differently.

WHY CHOOSE THIS?

CSS is transparent and fast, and over 97 rolling origins (section 6) its forecasts were only slightly less accurate than MLE's.

WHY NOT THIS? (TRADEOFFS)

Near –1 (as THETA is for CO2), CSS is measurably biased; for final forecasts the exact likelihood is the safer choice.

6. Rolling origins: many forecasts, not one

DEFINITION — start here, no prior knowledge assumed

One test period can mislead. A **rolling origin** evaluation repeats the whole procedure from many starting months: fit on everything before the origin, forecast the next 24 months, record the errors, move the origin one month on. All origins here are in the training years, so the 2000–2001 test months stay untouched.

REAL-WORLD SCENARIO

Judging a weather forecaster on a single day is unfair; judging them on every day of a year is not.

ACTUAL OUTPUT

```
97 origins, 1990-01 to 1998-01; every forecast ends by 1999-12, so the 2000-2001 test months are untouched
time: CSS 7.3 s for all fits, statsmodels 51.2 s
       MAE (ppm), months ahead       1       6      12      24   bias at 24
           airline, ours (CSS)   0.264   0.451   0.715   1.099   -0.288
    airline, statsmodels (MLE)   0.253   0.442   0.682   1.028   -0.246
          same month last year   1.362   1.428   1.560   3.137   -3.137
  95% interval coverage (ours)   0.959   0.969   0.938   0.979
lowest coverage over all 24 horizons: 0.897, 18 months ahead
MLE is more accurate than CSS at 24 of 24 horizons, by little; both are far ahead of last year's value, which misses the
    rise
Actual output from this lesson's run.
```

What it means

Over 97 origins the airline model's error grows from 0.264 ppm one month ahead to 1.099 at 24; same month last year is far worse (1.362 to 3.137) because it misses CO2's rise. statsmodels' MLE fit is more accurate at 24 of 24 horizons, but only slightly (0.253 and 1.028). Our 95% intervals held up: coverage 0.959, 0.969, 0.938 and 0.979 at 1, 6, 12 and 24 months, with the lowest 0.897, 18 months ahead. The fits took 7.3 s for CSS and 51.2 s for statsmodels.

WHY CHOOSE THIS?

Rolling origins turn one noisy comparison into 97, and check the intervals as well as the forecasts.

WHY NOT THIS? (TRADEOFFS)

Overlapping forecasts are not independent, so 97 origins carry less information than 97 separate tests would.

7. Electrical-equipment orders: an ordinary period, and the crash

DEFINITION — start here, no prior knowledge assumed

The same airline model on the log orders, scored in percentage errors. First, rolling origins through an ordinary stretch (2001–2007, 12 months ahead). Then two 24-month tests: through the 2008–09 crash, and over the last two years of the data. For each, AIC also picks a model on the training months.

REAL-WORLD SCENARIO

A forecaster who learned from calm years is asked about a storm. The question is not whether they predict it but whether their stated uncertainty warns that they might be wrong.

ACTUAL OUTPUT

```
rolling origins 2001-01 to 2007-01 (73), log orders, mean absolute % error by months ahead:
                             1       6      12
        airline (CSS)   1.92%   4.39%   6.26%
  same month last year   5.31%   5.13%   4.83%
            last value  10.89%   7.58%   4.83%
the model wins one month ahead, where recent errors carry information; by 12 months ahead last year's value is better
the crash: train up to 2007-12, forecast 2008-01 to 2009-12; same month last year: 18.04%
  airline                     MAE 23.86%   outside the 95% interval: 13 of 24   worst 2009-09: actual -32.0% against the
      forecast
  AIC's choice (2, 0, 0, 1)    MAE 23.87%   outside the 95% interval: 13 of 24   worst 2009-09: actual -32.0% against the
      forecast
the last 24 months: train up to 2014-05, forecast 2014-06 to 2016-05; same month last year: 3.75%
  airline                     MAE  6.41%   outside the 95% interval:  0 of 24   worst 2016-01: actual +11.3% against the
      forecast
  AIC's choice (2, 2, 0, 1)    MAE  6.72%   outside the 95% interval:  0 of 24   worst 2016-01: actual +11.7% against the
      forecast
in the crash no method saw it coming and the intervals were far too narrow; in the quiet last two years last year's
value still beat both models, and AIC's larger model was no better than the airline model
Actual output from this lesson's run.
```

What it means

In the ordinary period the model wins clearly one month ahead (1.92% against 5.31% for same month last year), where recent errors carry information, and loses a year ahead (6.26% against 4.83%).

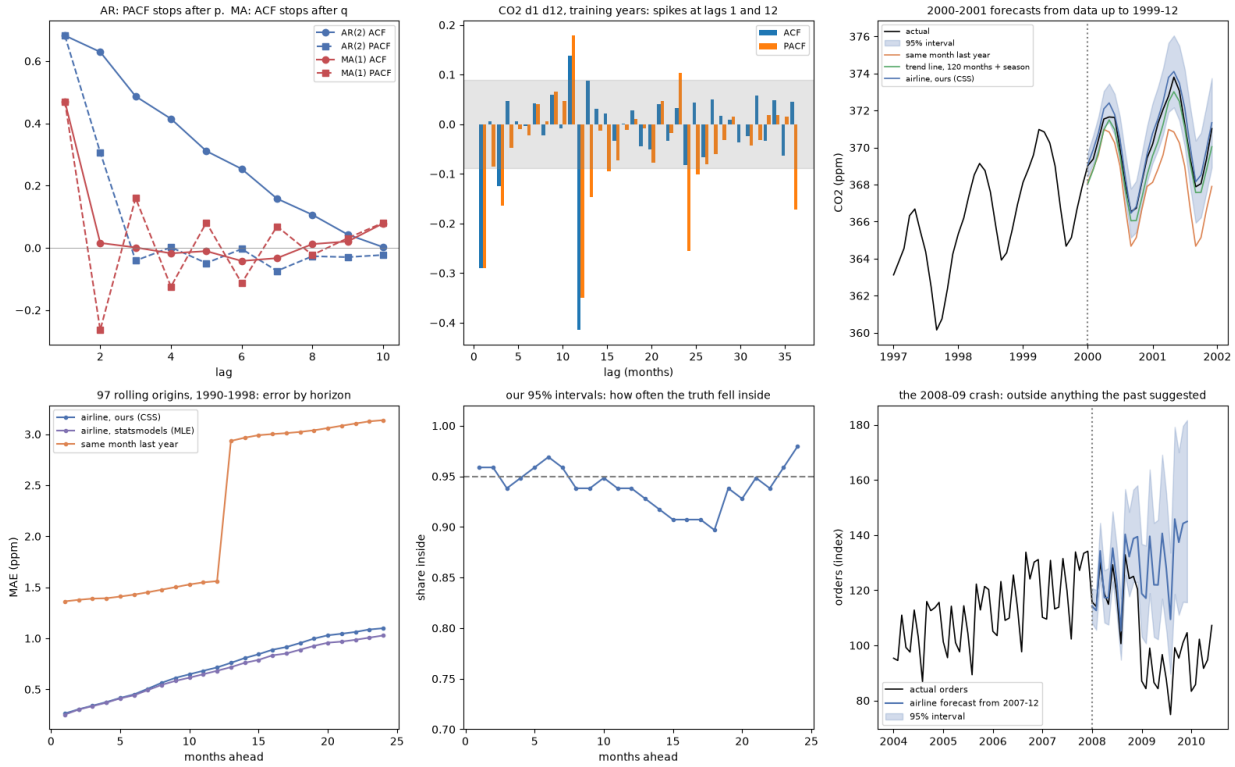
Through the crash, every method failed: the airline model missed by 23.86% on average, worse than same month last year (18.04%); 13 of 24 months fell outside its 95% interval, the worst being 2009-09, -32.0% below the forecast. AIC's choice did no better (23.87%). In the quiet last two years same month last year won again (3.75% against 6.41%), and AIC's larger model was slightly worse (6.72%).

WHY CHOOSE THIS?

ARIMA earns its keep at short horizons, and its intervals were well calibrated on CO2.

WHY NOT THIS? (TRADEOFFS)

Intervals assume the future behaves like the past; a crash breaks that, and on the orders a naive seasonal forecast was hard to beat a year ahead.



Top: AR and MA signatures in the ACF and PACF; CO2's d1 d12 ACF and PACF with the noise band; the 2000–2001 forecasts with our 95% interval. Bottom: rolling-origin error by horizon; interval coverage by horizon; the orders' forecast from 2007-12 against the crash.

Interview preparation — technical questions

Q: How do you choose p and q?

A: Read the ACF and PACF of the differenced series (an MA's ACF stops after q, an AR's PACF after p), then compare candidates by AIC on the same rows; for CO2 both pointed to the airline model.

Q: How are ARIMA forecasts turned back into the original units?

A: Multiply the AR polynomial by the differencing polynomials and run the recursion on the original series; it equalled undoing the differences by hand to 1.7e-13.

Q: How do you check that a fitted model is adequate?

A: Ljung-Box on the errors (here Q 28.52, p 0.159) and, more importantly, forecasts on data the fit never saw, from many origins.

Q: Why did statsmodels' fit differ from yours?

A: It maximises the exact likelihood; CSS sets early errors to 0, which a simulation showed pulls a seasonal MA near -1 towards 0 (40 of 40 runs).

Q: When would a seasonal naive forecast beat ARIMA?

A: When the series has little short-term memory to exploit at the horizon asked: for the orders 12 months ahead, 4.83% against 6.26%.

Interview preparation — theoretical questions

Q: What is the difference between AR and MA terms?

A: AR terms use earlier values, so a shock's influence decays gradually; MA terms use earlier errors, so a shock lasts exactly q steps.

Q: Why do forecast intervals widen with the horizon?

A: Each step adds an unknown future shock; the psi weights say how much each contributes (for CO2, 0.64 ppm one month ahead, 2.40 at 24).

Q: Why is the airline model so common for monthly data?

A: Two differences and two MA terms capture a drifting trend and a slowly changing season with only two coefficients.

Q: What can no ARIMA model forecast?

A: A break with the past: the 2008-09 crash fell outside the 95% interval in 13 of 24 months.

Key takeaway and what's next

> **KEY TAKEAWAY**
>
> ARIMA models what differencing leaves behind: memory in the values (AR) and in the errors (MA), at lag 1 and lag 12. Written from scratch it is a recursion, a least-squares solver and a polynomial product, and for the same coefficients it matches statsmodels' forecasts to 2.2e-07. For CO2 the airline model beat every simple forecast and its intervals were honest; for the orders it helped a month ahead, not a year ahead, and nothing foresaw the crash. Tomorrow's exponential smoothing is a different route to similar forecasts, compared on the same months.

- Forecasts match statsmodels' Kalman filter for the same coefficients (2.2e-07).
- AIC on the training years chose the airline model; 2000–2001 error 0.412 ppm against Day 147's best 0.591.
- CSS's seasonal coefficient -0.6902 against MLE's -0.8480; a simulation shows CSS biased towards 0 in 40 of 40.
- Rolling origins: 95% interval coverage between 0.897 and 0.979 at every horizon.
- Orders: 1.92% one month ahead, but the crash left 13 of 24 months outside the interval.

Suggested watch order, if starting from zero

- 1. ARIMA & SARIMA Made Easy: Forecasting for Beginners (Prof. Ryan Ahmed) — ARIMA and seasonal ARIMA explained for beginners, with a forecasting example.
- 2. Coding ARMA Model: Time Series Talk (ritvikmath) — an ARMA model coded and fitted step by step, to compare with today's from-scratch version.

Day 150 preview

- **Exponential smoothing.** Simple, Holt and Holt-Winters smoothing from scratch: forecasts as weighted averages of the past, with weights that decay geometrically. They are compared with today's ARIMA on the same CO2 holdout and the same rolling origins, with every setting chosen on earlier data only.